



Lab 04 – Worksheet

Name: Abdul Moiz Ahmed	ID: aa08811	Section: T4
------------------------	-------------	-------------

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Provide appropriately commented codes.

```
#n=5
addi x10, x0, 5

#call the factorial function
jal x1, fact

#move result to x11 to print
addi x11, x10, 0

#print
addi x10, x0, 1
ecall

#skip the function after returning
jal x0, exit

#listing 3.2 code
fact:
    addi sp, sp, -8    # adjust stack for 2 items
    sw x1, 4(sp)      # save return address
    sw x10, 0(sp)      # save argument n

    addi x5, x10, -1   # x5 = n - 1
    bge x5, x0, L1     # if (n - 1) >= 0, go to L1

    addi x10, x0, 1    # return 1
    addi sp, sp, 8     # pop stack
    jalr x0, 0(x1)     # return
```

L1:



```
addi x10, x10, -1    # argument = n - 1
jal x1, fact         # recursive call

addi x6, x10, 0      # save result of fact(n-1)
lw x10, 0(sp)        # restore original n
lw x1, 4(sp)         # restore return address
addi sp, sp, 8       # pop stack

mul x10, x10, x6     # n * fact(n-1)
jalr x0, 0(x1)       # return
```

exit:

**Add snip of relevant Registers / Memory locations where you think results are obtained. Make sure the irrelevant area of snip is cropped. Make sure the irrelevant area of snip is cropped.*



```
x07 (t2)    = 0x00000000
x08 (s0)    = 0x00000000
x09 (s1)    = 0x00000000
x10 (a0)    = 0x00000078
x11 (a1)    = 0x00000078
x12 (a2)    = 0x00000000
-----
x13 (a3)    = 0x00000000
```

Comments



We tested the factorial function with $n = 5$ and it returned 120 as the answer which is 0x78 in hexadecimal. The result is stored in register x10 and was also copied to x11 and during each recursive call the stack stores the return address x1 and the current value of n (x10). The stack pointer is decreased by 8 bytes for each call and restored as each recursive call returns

Task 2

Provide appropriately commented codes

```
#num=5
addi x10, x0, 5

#call ntri function
jal x1, ntri

#copy result to x11 to print
addi x11, x10, 0

#print
addi x10, x0, 1
ecall

jal x0, exit

ntri:
    #space for ra and num
    addi sp, sp, -8
    #save ra and num (current)
    sw x1, 4(sp)
    sw x10, 0(sp)

    #x5=1
    addi x5, x0, 1

    #if num <= 1 then go to base case
    ble x10, x5, base
    addi x10, x10, -1

    #recursive call
    jal x1, ntri
```



```
# save result of ntri(num - 1) and restore original num  
addi x6, x10, 0  
lw x10, 0(sp)
```

```
#ra restore and stack  
lw x1, 4(sp)  
addi sp, sp, 8
```

```
#num+ntri(num-1)  
add x10, x10, x6
```

```
jalr x0, 0(x1)
```

base:

```
#return 1  
addi x10, x0, 1  
lw x1, 4(sp)  
addi sp, sp, 8  
jalr x0, 0(x1)
```

exit:

Add screenshot of your results

--



```
x09 (s1) = 0x00000000
```

```
x10 (a0) = 0x0000000F
```

```
x11 (a1) = 0x0000000F
```

```
x12 (a2) = 0x00000000
```

```
-----  
Starting program
```

```
15
```

Comments

We tested the function with num =5 and it returned 15 which is 0x0F in hexadecimal. for each recursive call the current value of num and the return address are stored on the stack and the stack pointer is decreased by 8 bytes for each call. When the base case num<= 1 is reached then the function returns 1 and the previous values and return addresses are restored



Task 3

Provide appropriately commented codes

```
#base add of array
addi x10, x0, 0x200

#len=4
addi x11, x0, 4

#store the array: 3 1 4 2
addi x5, x0, 3
sw x5, 0(x10)

addi x5, x0, 1
sw x5, 4(x10)
addi x5, x0, 4
sw x5, 8(x10)
addi x5, x0, 2
sw x5, 12(x10)
jal x1, bubble

jal x0, exit

bubble:
    # if a == 0, return
    beq x10, x0, return
    # if len == 0, return
    beq x11, x0, return
    #i=0
    addi x5, x0, 0
outer_loop:

    # if i >= len, sorting is complete
```



```
bge x5, x11, return
#j=i
addi x6, x5, 0

inner_loop:

    # if j >= len, move to next i
    bge x6, x11, next_i
    #find a[i] address
    slli x7, x5, 2
    add x7, x10, x7

    #find a[j] address
    slli x28, x6, 2
    add x28, x10, x28

    #load a[i]
    lw x29, 0(x7)

    #load a[j]
    lw x30, 0(x28)
    # if a[i] >= a[j], do not swap
    bge x29, x30, no_swap
    #swap a[i] and a[j]
    sw x30, 0(x7)
    sw x29, 0(x28)

no_swap:
    #j=j+1
    addi x6, x6, 1

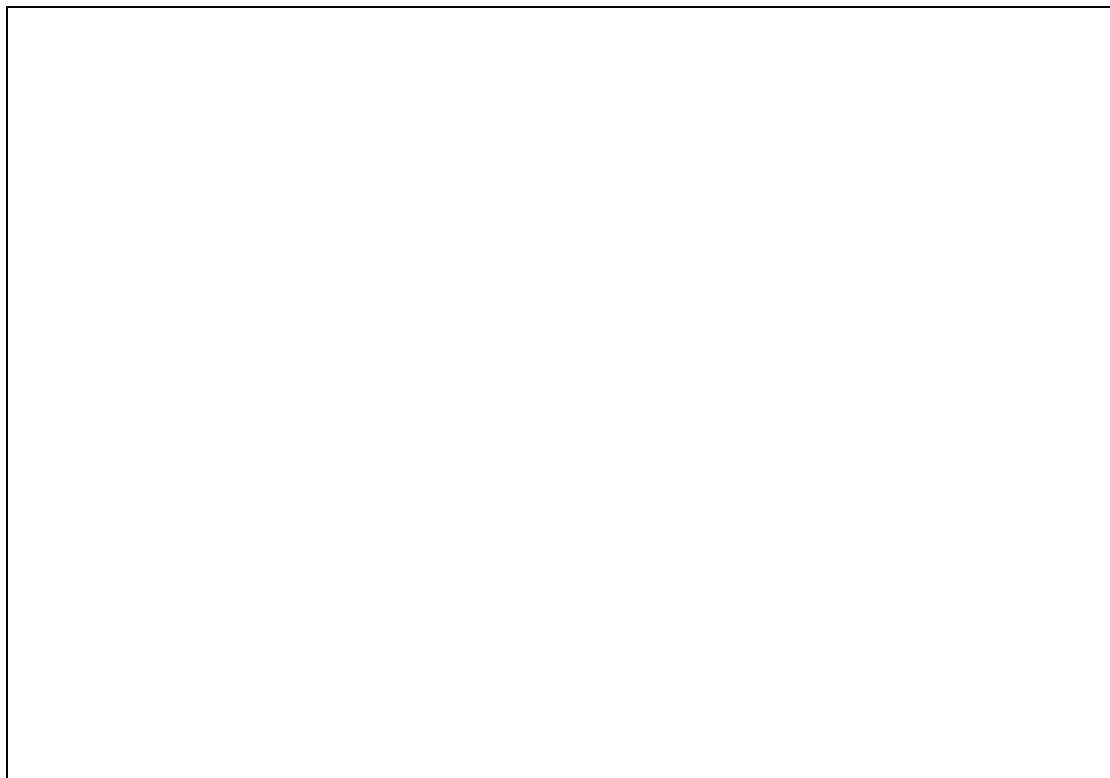
    #repeat inner loop
    jal x0, inner_loop

next_i:

    #i=i+1
    addi x5, x5, 1

    #repeat outer loop
    jal x0, outer_loop

return:
    jalr x0, 0(x1)
exit:
    jal x0, exit
```



Add screenshots of your results

0x00000210	00	00	00	00
0x0000020C	01	00	00	00
0x00000208	02	00	00	00
0x00000204	03	00	00	00
0x00000200	04	00	00	00
0x000001FC	00	00	00	00

Comments:

We tested the bubble sort function using the array [3, 1, 4, 2] and after execution, the values stored in memory became [4, 3, 2, 1] which showed that the array was sorted in descending order. we used two loops to compare array elements and swaps them whenever $a[i] < a[j]$



Task 4

Provide appropriately commented codes

--



```
#a=3
addi x10, x0, 3

#b=4
addi x11, x0, 4
#call func
jal x1, calculate

#copy ot x11
addi x11, x10, 0

#proint int
addi x10, x0, 1
ecall

jal x0, exit

calculate:
    #make space on stack and save ra
    addi sp, sp, -4
    sw x1, 0(sp)
#call sum, square fuctions
    jal x1, sum
    jal x1, square
#restore original ra and stack
    lw x1, 0(sp)
    addi sp, sp, 4

#return
    jalr x0, 0(x1)
```



sum:

#x10=a+ b

add x10, x10, x11

#return

jalr x0, 0(x1)

square:

#x10=x10*x10

mul x10, x10, x10

jalr x0, 0(x1)

exit:

jal x0, exit



Add screenshots of your results

```
x09 (s1) = 0x00000000
x10 (a0) = 0x00000001
x11 (a1) = 0x00000031
x12 (a2) = 0x00000000
```

Problems Output Debug Console Ter

Starting program C:\Users\abdu1\O

49

Comments:

For this task, we calculate $(a + b)^2$ using nested procedures. For $a = 3$ and $b = 4$ the sum function calculates $3 + 4 = 7$ and the square function calculates $7 \times 7 = 49$. The final result is stored in x11 as 0x31 and printed as 49 in the terminal. The calculate function saves its ra on the stack before calling the nested procedures and restores it





Lab 04 – Nest Procedures and Sorting Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	AR 7 Report Submission & Github
Task 1	/10	/05	/10 & /10
Task 2	/10	/10	
Task 3	/10	/10	
Task 4	/15	/10	
Total Points	/100 Points		
CLO Mapped	CLO 2		

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.



			Report is submitted after due date.		
--	--	--	--	--	--